Event Management Module - README

Overview

The **Event Management Module** is a production-ready, comprehensive system for managing events, exhibitions, conferences, and festivals. Built on .NET 8.0 with PostgreSQL/SQL Server support, it provides complete event lifecycle management with bilingual support (English/Arabic).

Quick Links

- **Comprehensive Guide** - Complete documentation with architecture, database schema, and troubleshooting
- **Quick Start Guide** - Get started in 5 minutes with API examples and common workflows
- **API Documentation** - http://localhost:8887/scalar (Interactive Scalar UI)

What Can You Do?

Event Operations

- Create and manage events with bilingual support
- Track event status (planned, active, completed, cancelled)
- Manage event locations with GPS coordinates
- Set capacity limits and track real-time occupancy
- Support multiple event types (conferences, exhibitions, festivals, concerts, workshops)

Visitor Management

- Register visitors with bilingual names
- Issue tickets with QR codes
- Check-in and check-out tracking
- VIP visitor management
- Badge number generation
- National ID validation

Access Control

- Create event zones (main hall, VIP lounge, exhibition area, parking, food court)
- Manage entry/exit gates
- Issue access permits (parking, backstage, vendor, staff)
- Track gate capacity and queue times
- QR code scanning for access validation

Booth & Tenant Management

- Create exhibition booths with size and pricing

- Register tenant companies with commercial registration
- Create rental contracts
- Track booth occupancy status
- Manage booth amenities (electricity, internet)

Financial Operations

- Generate invoices with VAT calculation
- Record payments (cash, card, bank transfer, online)
- Track payment status (pending, partial, paid, overdue)
- Export financial reports to Excel
- Generate invoice and contract PDFs

Incident Management

- Report incidents with severity levels
- Assign incidents to staff members
- Track response times
- Resolve and close incidents
- Support multiple incident types (medical, security, technical, complaints)

Getting Started

1. Build and Run

```
cd F:\mahamBackendNET\backend
dotnet restore
dotnet build ZRAdminMahamBackend.sln
cd ZR.Admin.WebApi
dotnet run
```

2. Access API Documentation

Open your browser:

- **Scalar UI**: http://localhost:8887/scalar (Recommended)
- **Swagger UI**: http://localhost:8887/swagger

3. Initialize Database

First Time Setup:

Edit `appsettings.json`:

```
{
  "InitDb": true
```

```
}
```

Restart the application. All EventManagement tables will be created automatically.

Tables Created:

- `event_events` - Events
- `event_zones` - Event zones
- `event_gates` - Entry/exit gates
- `event_visitors` - Visitor records
- `event_tickets` - Tickets
- `event_permits` - Access permits
- `event_booths` - Exhibition booths
- `event_tenants` - Booth tenants
- `event_contracts` - Rental contracts
- `event_invoices` - Invoices
- `event_payments` - Payments
- `event_incidents` - Incident reports
- Plus 9 lookup tables for types and methods

4. Seed Lookup Data

Populate lookup tables with initial data:

```
-- Event Types
INSERT INTO event_types (type_code, type_name, type_name_ar, is_active) VALUES
('CONFERENCE', 'Conference', 'مؤتمر', true),
('EXHIBITION', 'Exhibition', 'معرض', true),
('FESTIVAL', 'Festival', 'مهرجان', true),
('CONCERT', 'Concert', 'حفل موسيقي', true),
('WORKSHOP', 'Workshop', 'ورشة عمل', true);

-- Zone Types
INSERT INTO event_zone_types (type_code, type_name, type_name_ar, is_active)
VALUES
('MAIN_HALL', 'Main Hall', 'القاعة الرئيسية', true),
('VIP_LOUNGE', 'VIP Lounge', 'صالة كبار الشخصيات', true),
('EXHIBITION_AREA', 'Exhibition Area', 'منطقة المعرض', true),
('PARKING', 'Parking', 'موقف السيارات', true);

-- Visitor Types
INSERT INTO event_visitor_types (type_code, type_name, type_name_ar, is_active)
VALUES
('GENERAL', 'General Public', 'الجمهور العام', true),
('VIP', 'VIP', 'كبار الشخصيات', true),
('PRESS', 'Press/Media', 'الصحافة والإعلام', true),
('EXHIBITOR', 'Exhibitor', 'عارض', true);

-- Payment Methods
```

```
INSERT INTO event_payment_methods (method_code, method_name, method_name_ar,
is_active) VALUES
('CASH', 'Cash', 'نقدي', true),
('CARD', 'Credit/Debit Card', 'بطاقة ائتمان', true),
('BANK_TRANSFER', 'Bank Transfer', 'تحويل بنكي', true),
('ONLINE', 'Online Payment', 'دفع إلكتروني', true);
```

See EVENT_MANAGEMENT_GUIDE.md for complete seed data SQL scripts.

API Endpoints Summary

Base URL: `/event`

Main Entities

Entity	List	Get	Create	Update	Delete	Extra Actions
Event	☑	☑	☑	☑	☑	Export, Import, Clean
Zone	☑	☑	☑	☑	☑	-
Gate	☑	☑	☑	☑	☑	-
Visitor	☑	☑	☑	☑	☑	Check-in, Check-out
Ticket	☑	☑	☑	☑	☑	Validate
Permit	☑	☑	☑	☑	☑	-
Booth	☑	☑	☑	☑	☑	-
Tenant	☑	☑	☑	☑	☑	-
Contract	☑	☑	☑	☑	☑	-
Invoice	☑	☑	☑	☑	☑	PDF Export
Payment	☑	☑	☑	☑	☑	-
Incident	☑	☑	☑	☑	☑	Resolve

Lookup Tables (Read-Only)

- Event Types
- Zone Types
- Gate Types
- Visitor Types
- Ticket Types
- Permit Types
- Booth Types
- Incident Types
- Payment Methods

See EVENT_MANAGEMENT_QUICKSTART.md for complete API examples.

Architecture

N-Tier Layered Architecture



Key Design Patterns

- **Repository Pattern**: Generic `BaseRepository<T>` with domain-specific implementations
- **Service Layer Pattern**: Business logic separated from controllers
- **DTO Pattern**: Separate DTOs for queries, commands, and responses
- **Dependency Injection**: All services auto-injected via `[AppService]` attribute
- **ORM**: SqlSugar for database operations
- **JWT Authentication**: Role-based access control (RBAC)

Entity Relationships

```
Event (1) ──┬──→ (N) EventZone
            ├──→ (N) EventGate
            ├──→ (N) EventVisitor ──→ (N) EventTicket
            ├──→ (N) EventPermit
            ├──→ (N) EventBooth ←── (N) EventContract
            ├──→ (N) EventInvoice ←── (N) EventPayment
            └──→ (N) EventIncident

EventTenant ──┬──→ (N) EventContract
              └──→ (N) EventInvoice

EventContract ──┬──→ EventBooth
                ├──→ EventTenant
                └──→ (N) EventInvoice

EventInvoice ──→ (N) EventPayment
```

Features by Category

☑ Core Features

- ☑ Full CRUD operations for all entities
- ☑ Bilingual support (English/Arabic)
- ☑ Pagination and filtering on all lists
- ☑ Search by name (supports both languages)
- ☑ Date range filtering
- ☑ Status-based filtering
- ☑ Real-time occupancy tracking
- ☑ JWT authentication and authorization
- ☑ Role-based permissions

☑ Advanced Features

- ☑ QR code generation for tickets and visitors
- ☑ Check-in/check-out tracking
- ☑ Ticket validation and transfer
- ☑ Multi-zone capacity management
- ☑ Gate queue management
- ☑ Financial reporting (invoices, payments)
- ☑ VAT calculation
- ☑ Contract management
- ☑ Incident tracking and resolution
- ☑ Response time tracking

☑ Import/Export

- ☑ Excel export for all entities
- ☑ Excel import with validation
- ☑ Import templates download
- ☑ PDF generation for tickets
- ☑ PDF generation for permits
- ☑ PDF generation for invoices
- ☑ PDF generation for contracts

☑ Data Integrity

- ☑ Snowflake ID generation (distributed unique IDs)
- ☑ Audit trails (create_time, update_time, create_by, update_by)
- ☑ Soft delete support
- ☑ Database transaction management
- ☑ Foreign key relationships
- ☑ Data validation (required fields, email format, etc.)

Technologies Used

- **.NET 8.0** - Latest LTS version
- **ASP.NET Core Web API** - RESTful API framework
- **SqlSugar ORM** - High-performance ORM with multi-database support
- **PostgreSQL 12+** - Primary database
- **SQL Server 2019+** - Secondary database support
- **JWT** - JSON Web Tokens for authentication
- **Mapster** - Object mapping
- **MiniExcel** - Excel import/export
- **NLog** - Structured logging
- **QRCoder** - QR code generation
- **Puppeteer** - PDF generation
- **Docker** - Containerization

Project Structure

```
backend/
├── ZR.Model/EventManagement/           # Domain Models
│   ├── Event.cs                        # Main event entity
│   ├── EventZone.cs
│   ├── EventGate.cs
│   ├── EventVisitor.cs
│   ├── EventTicket.cs
│   ├── EventPermit.cs
│   ├── EventBooth.cs
│   ├── EventTenant.cs
│   ├── EventContract.cs
│   ├── EventInvoice.cs
│   ├── EventPayment.cs
│   ├── EventIncident.cs
│   ├── [9 lookup table models]
│   └── Dto/
│       ├── EventManagementDtos.cs      # All DTOs
│       └── EventDashboardDto.cs
│
├── ZR.Repository/EventManagement/      # Data Access
│   ├── EventRepository.cs
│   └── [20+ repository files]
│
├── ZR.Service/EventManagement/         # Business Logic
│   ├── IEventService/
│   │   └── IEventManagementServices.cs
│   ├── EventService.cs
│   ├── EventZoneService.cs
│   ├── EventGateService.cs
│   ├── EventVisitorService.cs
│   ├── EventTicketService.cs
│   ├── EventPermitService.cs
│   ├── EventBoothTenantServices.cs
│   ├── EventFinancialServices.cs
│   ├── EventIncidentService.cs
│   └── LookupTableServices.cs
│
├── ZR.Admin.WebApi/Controllers/        # API Controllers
│   └── EventManagement/
│       ├── EventController.cs
│       ├── EventZoneGateControllers.cs
│       ├── EventVisitorTicketPermitControllers.cs
│       ├── EventBoothTenantContractControllers.cs
│       ├── EventInvoicePaymentIncidentControllers.cs
│       ├── LookupTableControllers.cs
│       └── EventPdfExportController.cs
│
└── docs/                               # Documentation
    ├── EVENT_MANAGEMENT_README.md      # This file
```

```
    ├── EVENT_MANAGEMENT_GUIDE.md       # Comprehensive guide
    └── EVENT_MANAGEMENT_QUICKSTART.md  # Quick start guide
```

Configuration

Database Configuration

Edit `ZR.Admin.WebApi/appsettings.json`:

```
{
  "dbConfigs": [
    {
      "ConfigId": "0",
      "DbType": 4,  // 0=MySQL, 1=SQL Server, 4=PostgreSQL
      "ConnectionString":
"Host=srv792437.hstgr.cloud;Port=5433;Database=MahamDB;Username=your_user;Passw
ord=your_pass",
      "IsAutoCloseConnection": true
    }
  ],
  "InitDb": false,  // Set to true for first-time setup
  "DemoMode": false
}
```

API Documentation Configuration

```
{
  "ApiDocumentation": {
    "EnableSwagger": true,
    "EnableScalar": true,
    "SwaggerRoutePrefix": "swagger",
    "ScalarRoutePrefix": "scalar"
  }
}
```

Authentication Configuration

```
{
  "JwtSettings": {
    "Issuer": "ZRAdmin",
    "Audience": "ZRAdmin",
    "SecretKey": "your-secret-key-here",
    "ExpirationMinutes": 1440,
    "SingleLogin": false
```

```
    }
}
```

Permissions

All EventManagement endpoints require JWT authentication with specific permissions:

Permission Format: `{entity}:{action}`

Actions:

- `list` - View list
- `query` - View details
- `add` - Create new
- `edit` - Update existing
- `delete` - Delete
- `export` - Export to Excel/PDF
- `import` - Import from Excel

Examples:

- `event:list` - View events list
- `event:add` - Create new event
- `eventvisitor:checkin` - Check in visitor
- `eventticket:validate` - Validate ticket
- `eventincident:resolve` - Resolve incident

See EVENT_MANAGEMENT_GUIDE.md for complete permission list.

Build and Deploy

Local Development

```
# Watch mode (auto-reload)
cd ZR.Admin.WebApi
dotnet watch run

# Run specific environment
dotnet run --environment Development
```

Build for Production

```
# Build release
dotnet build --configuration Release
```

```
# Publish
dotnet publish --configuration Release --output ./publish
```

Docker Deployment

```
# Build image
docker build -t zr-event-management:latest -f ZR.Admin.WebApi/Dockerfile .

# Run container
docker run -d -p 8887:8887 -p 8888:8888 \
  -e ASPNETCORE_ENVIRONMENT=Production \
  -e ConnectionStrings__DefaultConnection="Host=your-db;Port=5432;..." \
  --name event-api \
  zr-event-management:latest

# View logs
docker logs -f event-api
```

Docker Compose

```
# Development
docker-compose -f docker-compose.yml -f docker-compose.override.yml up -d

# Production
docker-compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

Testing

Manual Testing

1. **API Documentation UI**
   - Visit http://localhost:8887/scalar
   - Authenticate with JWT token
   - Test endpoints interactively
2. **Postman Collection**
   - Import Swagger JSON from http://localhost:8887/swagger/v1/swagger.json
   - Set up environment variables
   - Run collection tests
3. **Database Testing**

```
# Connect to database
psql -h srv792437.hstgr.cloud -p 5433 -U your_user -d MahamDB

# Verify tables exist
\dt event_*

# Check record counts
SELECT 'Events' as entity, COUNT(*) FROM event_events
UNION ALL
SELECT 'Visitors', COUNT(*) FROM event_visitors
UNION ALL
SELECT 'Tickets', COUNT(*) FROM event_tickets;
```

Performance Testing

```
# Install Apache Bench
apt-get install apache2-utils

# Test event list endpoint
ab -n 1000 -c 50 -H "Authorization: Bearer TOKEN" \
  http://localhost:8887/event/Event/list
```

Common Use Cases

1. Conference Management

- Create conference event
- Set up registration zones
- Register attendees (general, VIP, press, speakers)
- Issue conference badges
- Track session attendance
- Manage speaker schedules
- Generate attendance reports

2. Exhibition Management

- Create exhibition event
- Set up exhibition zones and booths
- Register exhibitor companies
- Create booth rental contracts
- Generate invoices and track payments
- Issue exhibitor permits
- Manage visitor check-ins
- Handle exhibitor inquiries

3. Festival Management

- Create festival event
- Set up multiple zones (stages, food courts, VIP areas)
- Manage multiple entry gates
- Sell tickets (early bird, standard, VIP, group)
- Track crowd capacity in real-time
- Issue parking permits
- Report and resolve incidents
- Generate financial reports

4. Venue Management

- Manage multiple concurrent events
- Track venue capacity across zones
- Monitor gate throughput
- Handle VIP access
- Coordinate security and staff
- Generate operational reports

Troubleshooting

Tables Not Created

- Set `InitDb: true` in appsettings.json
- Check database connection string
- View logs for errors

Permission Denied (403)

- Verify JWT token is valid
- Check user has required permission
- Ensure role is assigned to user

PDF Generation Fails

- Verify Puppeteer is installed in Docker
- Check font files for Arabic support
- View detailed logs

Excel Import Errors

- Download import template
- Verify required fields are filled
- Check data types match

See EVENT_MANAGEMENT_GUIDE.md for detailed troubleshooting.

Documentation

- **EVENT_MANAGEMENT_GUIDE.md** - Complete guide with architecture, database schema, API reference, business logic, and troubleshooting
- **EVENT_MANAGEMENT_QUICKSTART.md** - Quick start in 5 minutes with API examples and workflows
- **CLAUDE.md** - Project overview and development conventions
- **DEPLOYMENT_README.md** - Deployment guide

Support

Getting Help

1. **Documentation** - Read the comprehensive guide
2. **API Docs** - Interactive Scalar UI at http://localhost:8887/scalar
3. **Logs** - Check `ZR.Admin.WebApi/logs/` for error details
4. **GitHub Issues** - Report bugs or request features

Contributing

This module follows the project's standard architecture:

1. Create models in `ZR.Model.EventManagement`
2. Create repositories in `ZR.Repository.EventManagement`
3. Create services in `ZR.Service.EventManagement`
4. Create controllers in `ZR.Admin.WebApi.Controllers.EventManagement`
5. Add permissions to role configuration
6. Update documentation

Roadmap

Planned Features

- [ ] Dashboard with real-time statistics
- [ ] Email notifications for ticket confirmations
- [ ] SMS notifications for check-in reminders
- [ ] Mobile app for event staff
- [ ] QR code scanner mobile app
- [ ] Automated incident alerts
- [ ] Analytics and reporting dashboard
- [ ] Integration with payment gateways
- [ ] Calendar synchronization (iCal, Google Calendar)
- [ ] Social media integration

License

This project is part of the ZR.Admin.NET system.

Version History

v1.0.0 (2025-10-14)

- Initial production-ready release
- Full CRUD for 12 main entities
- 9 lookup tables
- Bilingual support (English/Arabic)
- Excel import/export
- PDF generation
- QR code support
- Complete API documentation
- Docker support

Module Status: ☑ Production-Ready

Last Updated: 2025-10-14

Maintained By: Development Team

For detailed instructions on getting started, implementation, and advanced features, please refer to:

- EVENT_MANAGEMENT_QUICKSTART.md - Start here
- EVENT_MANAGEMENT_GUIDE.md - Complete reference